CS300 project one JAIDEN TRULL

Pseudocode 1: menu (within main function)

Choice = 0;

WHILE choice 9 is not chosen

OUTPUT menu options to screen

// choice 1: load data into data structure

// choice 2: Print all courses from data structure alphanumerically

// choice 3: Print course title and prerequisites for a given course

// choice 9: Exit program loop

GET choice from user

SWITCH for chosen choice

IF choice 1: call loadCSV() function

IF choice 2: call printSorted() function

IF choice 3: call search function from data structure class and print contents

ELSE IF course not found: print “course not found”

IF choice 9: Exit while loop

OUTPUT exit message

Pseudocode 2: printSorted function using Quick Sort

// helper function for successive partitions

Int partition(vector(Course<course>& courses, int begin, int end) {

SET low = 0

SET high = 0

SET middlePoint = low + (high – low) / 2

SET pivot = courses.at(middlePoint).courseNumber

```
SET done = false
WHILE done = false
WHILE courses.at(low).courseNumber < courses.at(pivot).courseNumber
low++
WHILE courses.at(high).courseNumber > courses.at(pivot).courseNumber
high--
IF low >= high // one or zero elements remain i.e. All courses are partitioned
done = true // exits loop
ELSE //swap low and high course elements
SET tempC = courses.at(low)
SET courses.at(low) = courses.at(high)
SET courses.at(high) = courses.at(tempC)
// Update low and high indexes
low++
high--
RETURN high // Upon loop completion
}

// sort given vector via quicksort
Void quickSort(vector<Course>& courses, int begin, int end) {
SET mid = 0

IF begin >= end
return // partition is already sorted
Mid = partition(courses, begin, end)
```

```
Quicksort(courses, begin, mid)

Quicksort(courses, mid, end)

}

// appends each data structure element to new vector

// sorts that vector

// prints out the sorted elements

Void printSorted(vector<Course>& courses) {

SET begin = 0

SET end = courses.size - 1

IF courses vector is empty

                FOR EACH element from given data structure

        APPHEND element to courses vector

QuickSort(courses, begin, end)

FOR EACH Course in courses vector

OUTPUT course title and course prerequisites

}
```

Evaluation

Vector: Big O value + advantages and disadvantages

The worse-case running time for the process of reading from the csv file into the vector data structure would consider the cost of each line of code and the number of times it will execute. Worst-case would occur when each line of the csv is properly formatted, and prerequisites courses are valid, such that no exceptions make the function exit early, and each csv line results in a course object with the maximum prerequisites (2). Assuming that the cost of each line to execute is 1 and there are N courses to be stored in the data structure, the Big O value would be the simplification of the total cost to execute. The presence of for loops and nested for loops when verifying the prerequisites courses indicates that the highest order term of the total would be N^2 after simplifying. Also, constant time operations become overshadowed and removed during Big O analysis due

to this higher term, resulting in the worst-case running time O(N^2). This means that for every N course during verification, the courses vector is passed through again for each prerequisite within a course object.

As a data structure, its implementation is simple and easy to understand. It can be limited in speed when given a large volume of input compared to the other two structures.

Hashtable: Big O value for reading from csv to data structure

Referring to the pseudocode for the hashtable, the is largely similar with the function first reading and parsing each line into a course element and then storing it inside a temporary vector. Then, for every N course during the verification process, each course's prerequisites are checked against the whole vector. This is the same verification process that resulted in N^2 before. After the element passes, it is inserted into the hashtable. The insert function includes a loop for traversing to the end of the chain and given a worst-case scenario that each course will be added to that chain, that loop will iterate N times. Due to the nested loop, N^2 is still the highest term and results in a worst-case running time of O(N^2).

The hash table insert function sets up a data structure that can allow for efficient search and remove operations. This one deals with collisions by chaining each additional node off the last. However, inputs that result in many nodes being placed in the same bucket make searching a little bit slower.

Binary Search Tree: Big O value for reading from csv to data structure

Referring to the pseudocode for the BST, it uses a temporary vector to store the csv data for verification in the same manner as the hashtable. However, the insert functionality uses the BST ordering property to add a node to the tree at the correct place. The worst-case scenario is where each processed element is less than or greater than the last, the tree would look more like a linked list and result in each tree recursing the line N times. The verification outweighs this and results in the same O(N^2) worst case running time.

The BST is an effective algorithm for searching nodes due to its ordering property. It has the capability to handle very large inputs without sacrificing the speed of accessing data. It does require a bit more space for each node due to having to store left and right pointers.

Final Recommendation

The Binary Search Tree offers a few extra advantages when compared to the other data structures. Its ordering property allows for efficient search operations and inherently allows a simpler operation for printing all courses in alphanumeric order. It was also a bit

easier to implement than the hashtable data structure. While hash tables can allow for encryption and decryption of the data, that may not be necessary for this. Referring to the Big O analysis, I believe the BST's verification code can be improved to result in a better worst case running time. Overall, the binary search tree appears that it may be the most effective option.